

Модули, пакеты и зависимости

Разделять код на модули, импортировать функции, понимать пакеты, зависимости, версии и виртуальные окружения.

DAY 16

MODULE

package

import

dependency

requirements

ЦЕЛЬ ДНЯ

Что ты сможешь после этого дня

Конкретные действия вместо списка терминов.

- 01 Разделять код на модули, импортировать функции, понимать пакеты, зависимости, версии и виртуальные окружения.
- 02 Объяснить минимум 5 терминов темы своими словами.
- 03 Прочитать показанный пример кода и предсказать результат.
- 04 Самостоятельно решить похожую задачу после разбора.

СВЯЗЬ

Новая тема не появляется из ниоткуда

Связываем сегодняшний шаг с уже построенной картой курса.

ВЧЕРА

Ошибки и отладка

Что уже есть в нашей модели.



СЕГОДНЯ

Модули, пакеты и зависимости

Вчера: Ошибки и отладка. Сегодня мы добавляем следующий слой: модули, пакеты и зависимости.

ЗАЧЕМ

Зачем мне это?

Каждая тема решает конкретную проблему.

ПРОБЛЕМА → ПОЛЬЗА

Один огромный файл быстро становится неудобным. Модули дают структуру, а фиксированные зависимости делают проект воспроизводимым на другом компьютере.

Вопрос себе

Какую реальную проблему решает сегодняшняя тема?

Ответь до следующего слайда.

Сначала — человеческий образ

Потом обязательно связываем его с настоящей техникой.

БЫТОВОЙ ОБРАЗ

Модуль — отдельный ящик с инструментами, пакет — набор связанных ящиков, requirements.txt — список того, какие внешние инструменты нужны проекту.

ПЕРЕВОД В IT

module

Python-файл, код которого можно импортировать

package

набор связанных Python-модулей

import

подключение доступного модуля в программу

Слой 1: module • package • import

Сначала смысл, затем профессиональный термин.

**module**

Python-файл, код которого можно импортировать

**package**

набор связанных Python-модулей

**import**

подключение доступного модуля в программу

Слой 2: dependency • requirements • version

Сначала смысл, затем профессиональный термин.

**dependency**

внешняя библиотека, от которой зависит проект

**requirements**

описание зависимостей проекта и их версий

**version**

идентификатор конкретного выпуска программы или пакета

Слой 3: namespace • semantic versioning

Сначала смысл, затем профессиональный термин.

namespace

область, связывающая имена с объектами

semantic versioning

схема версий вида major.minor.patch

Не заучивай формулировку. Придумай свой пример использования каждого термина.

Слой 4: virtual environment • standard library

Сначала смысл, затем профессиональный термин.

virtual environment

изолированное окружение зависимостей проекта

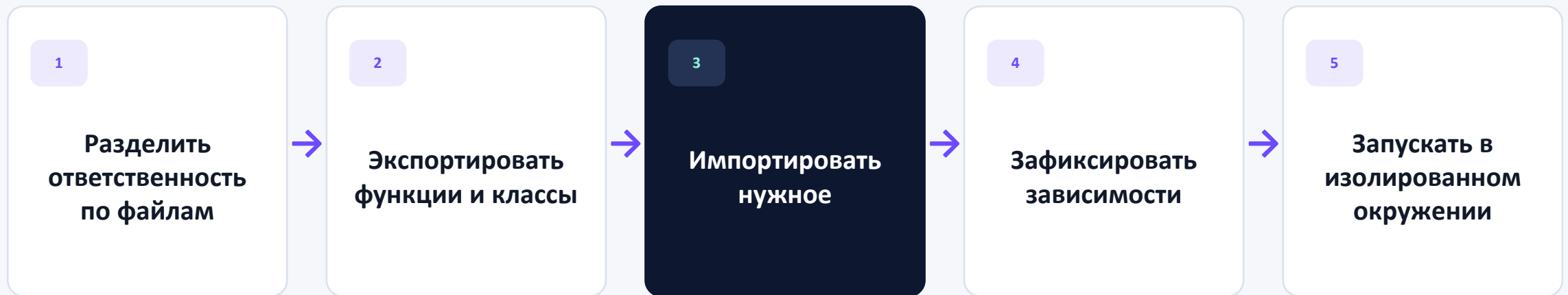
standard library

модули, поставляемые вместе с языком

Не заучивай формулировку. Придумай свой пример использования каждого термина.

Как это работает по шагам

Проследи движение слева направо.



Проговори эту цепочку своими словами, не подсматривая в текст.

Первый минимальный пример

Смотрим только на одну основную идею.



day16_step1

```
from pathlib import Path

file_path = Path("data") / "users.json"
print(file_path)
```

ЧТО ИСКАТЬ

- Где данные?
- Где действие?
- Где результат?
- Что изменится, если заменить одно значение?

Разбираем пример построчно

Код не должен оставаться магической конструкцией.

```
from pathlib import Path
```

Python выполняет эту строку сверху вниз.

```
file_path = Path("data") / "users.json"
```

Сохраняем значение или результат вычисления под понятным именем.

```
print(file_path)
```

Показываем результат, чтобы увидеть состояние программы.

РЕЗУЛЬТАТ

Код → выполнение → результат

Всегда связываем запись с тем, что реально получилось.



до запуска

```
from pathlib import Path  
  
file_path = Path("data") / "users.json"  
print(file_path)
```

РЕЗУЛЬТАТ

data/users.json # в Windows
разделитель может выглядеть

ИЗМЕНИЛИ

Теперь изменим пример

Сначала предскажи результат. Не запускай код сразу.



changed

```
import random  
print(random.randint(1, 3))
```



**Что появится в
результате?**

Сформулируй ответ словами.

ОТВЕТ

Сверяем прогноз

Не просто смотри ответ — объясни, почему он именно такой.

РЕЗУЛЬТАТ

случайное число 1, 2 или 3

ПОЧЕМУ

Python/система выполняет шаги в записанном порядке. Изменённые данные или действие дают новый результат.

Сравни с первым примером: что изменилось?

ОШИБКА

Как писать неправильно

Ошибка — отдельный учебный материал, а не повод её скрывать.



wrong

```
import random.py  
print(random.randint(1, 3))
```

СТОП

**Что здесь может сломаться
или ввести нас в
заблуждение?**

Сначала найди проблему сам.

Почему возникает проблема

Переводим техническую причину на обычный язык.

В `import` пишут имя модуля, а не имя файла с расширением `.py`. Стандартный модуль называется `random`.

Алгоритм отладки: симптом → конкретная строка/шаг → причина → минимальное исправление → повторная проверка.

ИСПРАВЛЕНИЕ

Исправленный вариант

Меняем только то, что действительно было причиной.

```
import random  
print(random.randint(1, 3))
```

fixed

ПРАВИЛО

Не переписывай всё решение. Найди минимальное осмысленное исправление.

Что изменилось после исправления?

Ответь без запуска.

- 1 Исчезла ли исходная причина проблемы?
- 2 Какой результат/поведение теперь ожидаем?
- 3 Как проверить это одним маленьким примером?

Сначала увидим полностью разобранный пример

Только после него будет самостоятельная практика.

ЗАДАЧА

Разнести расчёт цены по файлам `pricing.py` и `app.py` и импортировать функцию в приложение.

Не начинаем с кода. Сначала переводим задачу на человеческий язык.

КАК ДУМАТЬ

Как рассуждает программист

Разбиваем большую формулировку на маленькие действия.

- 1 В `pricing.py` создаём функцию.
- 2 Функция возвращает вычисление.
- 3 В `app.py` импортируем функцию по имени.
- 4 Вызываем её с аргументами.
- 5 Получаем тот же результат, но код разделён по ответственности.

Шаг 1: собираем основу

Не показываем 20 строк сразу — строим решение частями.



solution_part1

```
# pricing.py
def total(price, quantity):
    return price * quantity
```

Проверь

- Какие данные уже есть?
- Какой шаг плана реализован?
- Что ещё осталось добавить?

ИТОГОВЫЙ КОД

Шаг 2: итоговое решение

Теперь видим программу/запрос целиком.



```
# pricing.py
def total(price, quantity):
    return price * quantity

# app.py
# from pricing import total
print(total(10, 3))
```

solution_final

Проверяем результат

Сравниваем то, что ожидали, с тем, что получили.

РЕЗУЛЬТАТ

30

ГЛАВНОЕ

Задача была решена не потому, что мы «знали готовый код», а потому что сначала построили понятный план.

ПОПРОБУЙ САМ

Практика 1

Задача очень похожа на разобранную — но теперь первый ход за тобой.

Спроектируй структуру из `app.py`, `utils.py` и `data/` и объясни ответственность каждой части.

Сначала выпиши 2–5 шагов решения словами.

ПОДСКАЗКА

Подсказка — только направление

Попробуй ещё раз до просмотра готового решения.

app.py — входная точка, utils.py — переиспользуемая логика, data/ — файлы данных.

Вопрос себе: какие конструкции сегодняшнего дня нужны, а какие пока лишние?

Один из правильных вариантов

Сравни логику, а не количество строк.



practice_solution

```
# utils.py
def normalize_name(name):
    return name.strip().title()

# app.py
# from utils import normalize_name
print(normalize_name("  anna "))
```

Ожидаемый результат

Если твой код устроен иначе, но даёт правильное и понятное решение — это нормально.

РЕЗУЛЬТАТ

Anna

Сравни

- Результат
- Понятность имён
- Лишние шаги
- Обработку ошибок

Практика 2 — мини-проект

Соединяем несколько элементов дня в одну маленькую закончённую задачу.

ЗАДАНИЕ

Сформируй минимальный requirements.txt с фиксированной версией Flask.

Попробуй сначала без подсказки. Разрешено подсматривать в сегодняшние примеры.

РЕШЕНИЕ МИНИ-ПРОЕКТА

Разбор мини-проекта

Вот один рабочий путь.



mini_project

```
# requirements.txt  
Flask==3.1.2
```

РЕЗУЛЬТАТ ПРОЕКТА

Что должно получиться

Связываем итог с задачей и проверяем смысл.

РЕЗУЛЬТАТ

Одинаковая версия Flask будет у всех участников проекта

Объясни каждую важную строку/команду своими словами. Если не можешь — вернись к конкретному месту, а не перечитывай весь день.

Четыре ловушки новичка

Лучше увидеть их здесь, чем случайно закрепить как привычку.

01

Называть свой файл как стандартный модуль, например `random.py`.

02

Ставить пакеты глобально вместо `venv`.

03

Не фиксировать версии зависимостей.

04

Создавать циклические импорты между модулями.

Проверь себя: 1–2

Не листай дальше, пока не сформулируешь ответ.

- 1 Что такое «module» и какую проблему это понятие помогает описать?
- 2 Чем «module» отличается от «dependency»? Объясни без копирования определения.

Проверь себя: 3–4

Здесь проверяем понимание кода и процесса мышления.

- 3 Какой результат даёт первый минимальный пример и почему?
- 4 Какую ошибку/риск мы разобрали сегодня и какое минимальное исправление нужно?

ОТВЕТЫ

Ответы к вопросам 1–2

Формулировка может отличаться — важен смысл.

1 **module — Python-файл, код которого можно импортировать.**

2 **module: Python-файл, код которого можно импортировать. dependency: внешняя библиотека, от которой зависит проект.**

ОТВЕТЫ

Ответы к вопросам 3–4

Сверяем причинно-следственную связь.

3 **data/users.json # в Windows разделитель может выглядеть иначе**

4 **В import пишут имя модуля, а не имя файла с расширением .py. Стандартный модуль называется random.**

ИТОГ

Сегодня мы научились

Не перечисляем слова — фиксируем новые способности.

- ✓ Разделять код на модули, импортировать функции, понимать пакеты, зависимости, версии и виртуальные окружения.
- ✓ Читать и объяснять пример темы «Модули, пакеты и зависимости».
- ✓ Находить типичную ошибку и формулировать причину.
- ✓ Решать похожую задачу по шагам, а не угадывать готовый код.

ДАЛЬШЕ

Что будет завтра

Связываем сегодняшний результат со следующим шагом.

Файлы, JSON и CSV

Завтра: Файлы, JSON и CSV. Сегодняшняя модель станет опорой для следующего шага.

Сегодняшняя тема не заканчивается здесь — она станет инструментом в следующем дне.